hexens x ROYCO

Security Review Report for Royco

July 2026

Table of Contents

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for Royco, a perpetual risk-tranching protocol. This engagement covered major refactoring of the original code. The update includes splitting pools into 3 different tranches and introduces the liquidity tranche where holders can provide secondary liquidity for a premium.

Our security assessment was a full review of the new code, spanning a total of 1 week.

During our review, we did not identify any major severity vulnerabilities.

We did identify some minor severity vulnerabilities and code optimizations.

All reported issues were fixed by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

Review Led by

Kasper Zwijsen, Head of Audits

Scope

The analyzed resources are located on:

🔗 https://github.com/roycoprotocol/royco-day/tree/c006198ca5b829387c62ac5d9655cca928dca6ba

The issues described in this report were fixed in the following RP:

🔗 https://github.com/roycoprotocol/royco-day/pull/17

Changelog

8 July 2026	Audit start
17 July 2026	Initial report
28 July 2026	Revision received
6 August 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

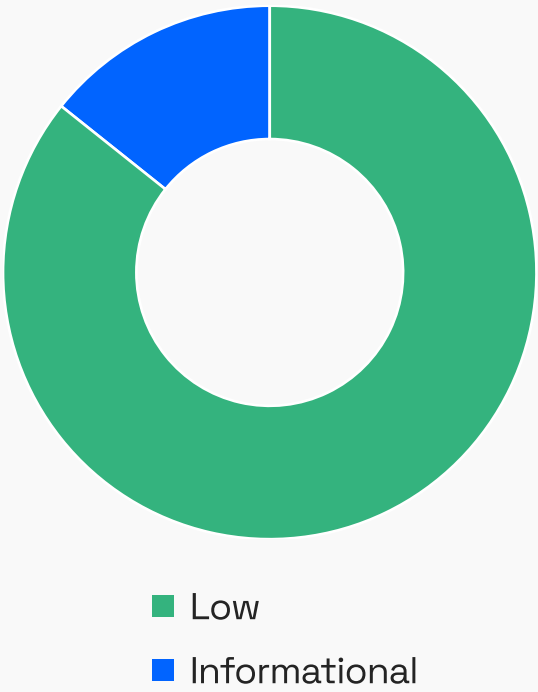
- **Issue Symbolic Codes**

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	0
Medium	0
Low	6
Informational	1
Total:	**7**



6. Weaknesses

This section contains the list of discovered weaknesses.

ROYCO5-1 | Historical LT premium can be assigned to the first LT depositor in zero-liquidity markets

Fixed

Severity:	Low	Probability:	Rare	Impact:	Low

Path:

src/libraries/logic/ValuationLogic.sol#L121

Description:

When a market is configured with **minLiquidityWAD == 0** and a non-zero **maxLTYieldShareWAD**, the accountant can still calculate an LT liquidity premium from senior yield. The LT premium rate is derived from liquidity utilization and capped by **maxLTYieldShareWAD** and there is no check here that disables the LT premium when the liquidity requirement is zero.

```
_twLTYieldShareAccruedWAD = Math.min(
    IYDM($.ltYDM)
        .previewYieldShare(
            initialMarketState, UtilizationLogic._computeLiquidityUtilization($.lastSTEffectiveNAV, $.minLiquidityWAD, $.lastLTRawNAV)
        ),
    $.maxLTYieldShareWAD
);

jtRiskPremium = stGain.mulDiv(_twJTYieldShareAccruedWAD, (elapsedSinceLastPremiumPayments * WAD), Math.Rounding.Floor);
ltLiquidityPremium = stGain.mulDiv(_twLTYieldShareAccruedWAD, (elapsedSinceLastPremiumPayments * WAD), Math.Rounding.Floor);
```

The premium is then minted as senior tranche shares held by the kernel for the LT, even if no LT shares exist yet.

```
    if (liquidityPremiumShares != 0) {
        IRoycoSeniorTranche(_immutables.seniorTranche).mintLiquidityPremiumShares(address(this), liquidityPremiumShares);
        $.ltOwnedSeniorTrancheShares += liquidityPremiumShares;
        IRoycoDayKernel(address(this)).attemptLiquidityPremiumReinvestment(type(uint256).max, _state.stEffectiveNAV, stTotalSupplyAfterMints);
    }
```

Those idle senior shares are included in the LT effective NAV.

```
    function _getLiquidityTrancheEffectiveNAV(
        IRoycoDayKernel.RoycoDayKernelState storage $,
        NAV_UNIT _stEffectiveNAV,
        uint256 _totalSeniorTrancheShares,
        uint256 _ltOwnedSeniorTrancheShares
    )
        internal
        view
        returns (NAV_UNIT ltEffectiveNAV)
    {
        // Get the value of LT's market-making inventory
        NAV_UNIT ltRawNAV = _getLiquidityTrancheRawNAV($);

        // If there are no held senior shares or no senior shares outstanding, the effective NAV is just the raw NAV
        if (_ltOwnedSeniorTrancheShares == 0 || _totalSeniorTrancheShares == 0) return ltRawNAV;

        // The LT effective NAV is the sum of the NAVs of its market-making inventory and its held ST shares
        return (ltRawNAV + _convertToValue(_ltOwnedSeniorTrancheShares, _totalSeniorTrancheShares, _stEffectiveNAV, Math.Rounding.Floor));
    }
```

Separately, when the LT total supply is zero, the first LT deposit mints shares 1:1 against the deposited value. Since there are no existing LT shares, that depositor becomes the only LT holder.

```
    function _convertToShares(NAV_UNIT _value, NAV_UNIT _totalValue, uint256 _totalSupply, Math.Rounding _rounding) internal pure returns (uint256 shares) {
        // With no shares outstanding the conversion is 1:1 with the value, mirroring the tranche's first mint
        if (_totalSupply == 0) return toUint256(_value);
        // When the total value is zero, assume all existing shares are backed by a single NAV unit so new depositors dilute the existing unbacked holders
        NAV_UNIT denominator = (_totalValue == ZERO_NAV_UNITS ? ONE_NAV_UNIT : _totalValue);
```

```
        // The overflow-free bind test, run before the fair-shares division:
        // fair > cap ⟺ value·(WAD − MAX_MINT_DILUTION_WAD) > denominator·MAX_MINT_DILUTION_WAD
        //            ⟺ ⌈value·(WAD − MAX_MINT_DILUTION_WAD) / MAX_MINT_DILUTION_WAD⌉ > denominator
        if (_value.mulDiv((WAD - MAX_MINT_DILUTION_WAD), MAX_MINT_DILUTION_WAD,
    Math.Rounding.Ceil) > denominator) {
            // The mint binds the clamp: pre-existing holders retain at least the complement of the max dilution
            return Math.mulDiv(_totalSupply, MAX_MINT_DILUTION_WAD, (WAD - MAX_MINT_DILUTION_WAD));
        }
        return _totalSupply.mulDiv(_value, denominator, _rounding);
    }
```

For example, consider a zero-min-liquidity market where ST/JT capital is seeded first and the LT remains empty. After senior yield accrues across syncs, the accounting logic may stage 100 NAV worth of LT premium as kernel-held senior shares before any LT shares exist. If a user then makes the first LT deposit with 1 NAV, the zero-supply bootstrap mints the initial LT shares 1:1 and makes that user the sole LT holder. The user's LT position would then include both the 1 NAV deposit and the previously staged 100 NAV premium.

Remediation:

- Enforce at initialization and in setters that **minLiquidityWAD == 0** cannot coexist with a non-zero **maxLTYieldShareWAD**.
- Alternatively, skip LT premium accrual while LT total supply is zero.
- If the premium should still accrue in that state, redirect it to senior holders instead of staging it for a zero-supply LT.

ROYCO5-2 | Zeroed senior recoveries are assigned to junior holders

Fixed

Severity:	Probability:	Impact:
Low	Rare	Medium

Path:

src/accountant/RoycoDayAccountant.sol:_previewSyncTrancheAccounting#L480-L533

Description:

The function **_previewSyncTrancheAccounting** is the main accounting path for senior and junior tranche NAV. The kernel calls it before deposits, redemptions, and any other syncs. It compares the new senior and junior raw NAVs against the last checkpoint, attributes the PnL to senior and junior effective NAV, then lets the rest of the redemption flow derive asset claims from those effective NAVs.

The function affects all senior and junior holders, but the bad outcome benefits junior holders. A junior holder, or someone who buys distressed junior shares, can trigger a normal sync or junior redemption after a severe loss and later recovery.

The intended behavior is that senior keeps priority on recoveries. The docs describe senior as the capital-protected tranche and state that senior has first claim on recoveries after a loss. The recovery waterfall also says senior tranche impermanent loss should be recovered first, then junior coverage losses, and only then should remaining gains be distributed as yield.

So if senior shares still exist, a later recovery in senior raw NAV should rebuild senior value first. For example, if the senior side was worth 100, was checkpointed at 0 after a severe loss, and later recovers to 50, that 50 should not become junior NAV just because the previous senior effective NAV was also 0. Senior shares were not burned by the sync, so there are still senior holders waiting on the recovery.

However, the zero raw NAV branch uses **stEffectiveNAV > 0** as a proxy for whether senior should receive the senior raw NAV delta:

```
int256 deltaSTClaimOnSTRawNAV = lastSTRawNAV == ZERO_NAV_UNITS
    ? (stEffectiveNAV > ZERO_NAV_UNITS ? deltaSTRawNAV : int256(0))
    : _attributeDeltaToClaimOnRawNAV(deltaSTRawNAV, stClaimOnSTRawNAV, lastSTRawNAV);
```

When both **lastSTRawNAV** and **stEffectiveNAV** are zero, this sets the senior attribution to zero. The recovered senior raw NAV is then left in the residual and becomes junior effective NAV:

```
deltaJTEffectiveNAV = (deltaSTRawNAV + deltaJTRawNAV) - deltaSTEffectiveNAV;
```

This contradicts the recovery waterfall. A total write-down can make senior effective NAV zero while senior shares still exist. The code then treats those live senior shares as if they do not exist. The comment says this avoids inflating NAV against zero ST shares, but the branch never checks ST share supply.

A simple trace shows the problem:

1. The market has senior and junior shares outstanding. The last checkpoint is **lastSTRawNAV = 100**, **lastJTRawNAV = 20**, **lastSTEffectiveNAV = 100**, and **lastJTEffectiveNAV = 20**.
2. The shared ST/JT conversion rate collapses to zero, or low enough that the quoted raw NAV floors to zero. A sync runs with **stRawNAV = 0** and **jtRawNAV = 0**. Junior effective NAV is wiped out first, then senior effective NAV is also reduced to zero. The senior and junior ERC20 shares still exist.
3. An attacker holds or buys junior shares after the write-down.
4. The conversion rate later recovers. The next raw NAVs are **stRawNAV = 50** and **jtRawNAV = 10**.
5. The attacker calls **RoycoVaultTranche.redeem(jtShares, attacker, attacker)**. This goes through **RoycoDayKernel.jtRedeem**, then **RedemptionLogic.jtRedeem**, then **AccountingSyncLogic._preOpSyncTrancheAccounting**, which calls **preOpSyncTrancheAccounting(50, 10)**.
6. Inside **_previewSyncTrancheAccounting**, **lastSTRawNAV == 0** and **stEffectiveNAV == 0**, so **deltaSTClaimOnSTRawNAV** is set to **0**. The whole **+60** raw NAV delta is booked as junior effective NAV.
7. **TrancheClaimsLogic._computeSTandJTClaimsOnRawNAVs** then decomposes the claims as **jtClaimOnSTRawNAV = 50** and **jtClaimOnJTRawNAV = 10**. The junior redemption path scales those claims by the attacker's junior share balance and withdraws the assets.

The result is that the senior-side recovery is redeemable by junior holders. Senior holders stay at zero effective NAV even though the senior assets recovered.

```
    function _previewSyncTrancheAccounting(
        NAV_UNIT _stRawNAV,
        NAV_UNIT _jtRawNAV,
        uint256 _twJTYieldShareAccruedWAD,
        uint256 _twLTYieldShareAccruedWAD
    )
        internal
        view
        returns (SyncedAccountingState memory state, MarketState initialMarketState, bool premiumsPaid,
NAV_UNIT jtCoverageImpermanentLossErased)
    {
```

```
        [..]
        int256 deltaSTClaimOnSTRawNAV = lastSTRawNAV == ZERO_NAV_UNITS
          ? (stEffectiveNAV > ZERO_NAV_UNITS ? deltaSTRawNAV : int256(0))
          : _attributeDeltaToClaimOnRawNAV(deltaSTRawNAV, stClaimOnSTRawNAV, lastSTRawNAV);
        int256 deltaSTClaimOnJTRawNAV = _attributeDeltaToClaimOnRawNAV(deltaJTRawNAV,
stClaimOnJTRawNAV, lastJTRawNAV);
        [..]
    }
```

Remediation:

Do not use zero senior effective NAV as proof that there are no live senior claims. Track senior impermanent loss explicitly, and pass senior share supply into the accounting sync so positive senior raw-NAV recovery is attributed to senior while senior shares are still outstanding.

ROYCO5-3 | Transfer whitelist blocks LT senior-share settlement into Balancer

Fixed

Severity:	Probability:	Impact:
Low	Rare	Medium

Path:

src/factory/templates/BalancerV3DeploymentTemplate.sol:_buildRoleBindings#L430-L466

Description:

The function **_buildRoleBindings()** builds the AccessManager bindings and initial role grants for a Balancer V3 market deployment. In a market that enables **ENFORCE_TRANCHE_WHITELIST_ON_TRANSFER**, these grants decide which addresses are allowed to receive tranche shares. The affected flow starts at deployment, then shows up later when a normal user calls an LT flow such as **depositMultiAsset()** with a nonzero senior asset leg, or when the protocol tries to reinvest LT liquidity premium.

In the expected setup, transfer-whitelist mode should block random recipients while still allowing the protocol's own settlement path to work. That means every protocol address that normally receives tranche shares has to be whitelisted. For LT Balancer adds, the kernel can mint senior shares to itself, and then those senior shares should be transferred into the Balancer Vault so the Vault debt can be settled.

The code already handles some internal recipients. The template grants the tranche LP roles to the kernel and protocol fee recipient, and the hook also skips transfers where the recipient is the kernel:

```
if (_to != address(0) && _to != address(this) && ENFORCE_TRANCHE_WHITELIST_ON_TRANSFER) {
    (bool isWhitelistedTrancheLP,) =
        IAccessManager(authority).canCall(_to, msg.sender, IRoycoVaultTranche.deposit.selector);
    require(_to != authority && isWhitelistedTrancheLP, ACCOUNT_NOT_WHITELISTED_TRANCHE_LP(_to));
}
```

However, the same template does not grant **ST_LP_ROLE** to the Balancer Vault. That matters because the whitelist check does not ask whether the transfer is part of a protocol-internal Balancer settlement. It only asks whether the recipient can call **deposit()** on the tranche token. Senior **deposit()** is bound to **ST_LP_ROLE**, so the Vault must hold that role if it can receive senior tranche shares.

The Balancer add-liquidity callback does exactly that transfer:

```
if (_seniorShares > 0) {
    IERC20(_venue.seniorTranche).safeTransfer(address(_venue.vault), _seniorShares);
    _venue.vault.settle(IERC20(_venue.seniorTranche), _seniorShares);
}
```

So a whitelist-enabled deployment can mint the senior shares to the kernel, but then fails when those shares are moved to the Balancer Vault. The result is not a direct loss of funds, but it does brick LT multi-asset deposits with a senior leg until an admin manually grants the missing role. Premium reinvestment hits the same missing recipient; that path catches the failure, so sync does not revert, but the premium stays idle instead of being put back into the Balancer venue.

An example failure path is:

1. A deployer creates a Balancer V3 market with **ENFORCE_TRANCHE_WHITELIST_ON_TRANSFER** enabled and uses the template's normal role grants.
2. The template grants **ST_LP_ROLE** to the kernel and protocol fee recipient, but not to **BALANCER_V3_VAULT**.
3. Alice calls **depositMultiAsset()** with **_stAssets > 0** and some quote assets.
4. **ltDepositMultiAsset()** mints the needed senior tranche shares to the kernel. This part passes because the hook skips **_to == address(this)**.
5. The kernel enters the Balancer add-liquidity path.
6. The Balancer callback tries to transfer the minted senior shares from the kernel to the Balancer Vault.
7. **RoycoVaultTranche._update()** calls **preTrancheBalanceUpdateHook()**, which checks whether the Vault can call **deposit()** on the senior tranche.
8. The Vault does not have **ST_LP_ROLE**, so the transfer reverts with **ACCOUNT_NOT_WHITELISTED_TRANCHE_LP**, and Alice's LT deposit cannot complete.

```
    function _buildRoleBindings(
        DeploymentResult memory _r,
        address _balancerHook,
        address _protocolFeeRecipient
    )
        internal
        view
        virtual
        returns (RoleBindings memory)
    {
        TargetBinding[] memory targets = new TargetBinding[](9);
        targets[0] = _trancheBinding(_r.seniorTranche, ST_LP_ROLE, ST_LP_ROLE, false);
        targets[1] = _trancheBinding(_r.juniorTranche, JT_LP_ROLE, JT_LP_ROLE, false);
        targets[2] = _trancheBinding(_r.liquidityTranche, PUBLIC_ROLE, LT_LP_ROLE, true);
```

```
        targets[3] = _kernelBinding(_r.kernel);
        targets[4] = _accountantBinding(_r.accountant);
        targets[5] = _balancerVaultBinding(address(BALANCER_V3_VAULT));
        targets[6] =
_balancerProtocolFeeControllerBinding(address(BALANCER_V3_VAULT.getProtocolFeeController()));
        targets[7] = _balancerHookBinding(_balancerHook);
        targets[8] = _kernelQuoterBinding(_r.kernel);

        // The kernel (coverage-neutral premium senior-share mint recipient) and the protocol fee recipient (ST/
JT/LT
        // fee-share mint recipient) must hold the tranche LP roles, otherwise a whitelist-enforcing market bricks
on
        // the first fee/premium mint when the tranche `_update` whitelist screen rejects them
        RoleGrant[] memory grants = new RoleGrant[](9);
        grants[0] = RoleGrant({ roleId: SYNC_ROLE, account: _r.accountant, executionDelay: 0 });
        grants[1] = RoleGrant({ roleId: BURNER_ROLE, account: _r.kernel, executionDelay: 0 });
        grants[2] = RoleGrant({ roleId: SYNC_ROLE, account: _balancerHook, executionDelay: 0 });
        grants[3] = RoleGrant({ roleId: ST_LP_ROLE, account: _r.kernel, executionDelay: 0 });
        grants[4] = RoleGrant({ roleId: JT_LP_ROLE, account: _r.kernel, executionDelay: 0 });
        grants[5] = RoleGrant({ roleId: LT_LP_ROLE, account: _r.kernel, executionDelay: 0 });
        grants[6] = RoleGrant({ roleId: ST_LP_ROLE, account: _protocolFeeRecipient, executionDelay: 0 });
        grants[7] = RoleGrant({ roleId: JT_LP_ROLE, account: _protocolFeeRecipient, executionDelay: 0 });
        grants[8] = RoleGrant({ roleId: LT_LP_ROLE, account: _protocolFeeRecipient, executionDelay: 0 });

        return RoleBindings({ targetBindings: targets, postInitGrants: grants });
    }
```

Remediation:

When tranche-transfer whitelist mode is supported, the deployment template should grant the senior LP role to the Balancer Vault, or to whatever venue settlement address receives senior tranche shares. This keeps the user whitelist strict while allowing the protocol's own Balancer settlement path to run.

ROYCO5-4 | Batch redemption execution can be blocked by a single non-executable request

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

./src/entrypoint/RoycoDayEntryPoint.sol

Description:

_validateRequestExecution reverts when a request has not yet become executable or when its oracle clock has not advanced. In **executeRedemptions**, requests are processed inside a loop, so a single failing request causes the entire batch to revert.

_validateRequestExecution contains hard require checks for both **executableAtTimestamp <= block.timestamp** and **ORACLE_CLOCK_NOT_ADVANCED**. **executeRedemptions** iterates through the batch without isolation, so any revert aborts execution of all requests in the batch. **requestRedemption** allows different tranches and different queued timestamps, making mixed-executable batches possible.

```
function _validateRequestExecution(uint256 _requestNonce, BaseRequest memory _baseRequest, address
_oracleClock) internal {
    // Ensure the request exists and the configured delay period has elapsed
    require(_baseRequest.executableAtTimestamp != 0 && _baseRequest.executableAtTimestamp <=
block.timestamp, INVALID_REQUEST(_requestNonce));
    // Ensure the tranche's oracle clock has observed an oracle update strictly after the request was queued
    require(
        _oracleClock == address(0) || (_pokeOracleClock(_baseRequest.tranche, _oracleClock) >
_baseRequest.queuedAtTimestamp),
        ORACLE_CLOCK_NOT_ADVANCED(_requestNonce)
    );
}
```

Remediation:

Consider replacing the revert path for non-executable requests with an early return or skip mechanism during batch execution so that executable requests can still be processed while invalid or not-yet-executable requests are ignored.

ROYCO5-5 | LT forfeiture ignores the idle liquidity premium

Fixed

Severity:	Probability:	Impact:
Low	Unlikely	Low

Description:

The entry point calculates LT redemption forfeiture using **convertToAssets**, which only reflects the BPT NAV and excludes idle liquidity premium (**ltOwnedSeniorTrancheShares**).

However, the payout always includes that premium (as idle senior shares, or inside the BPT once reinvested)

Scenario 1: Queue yield/premium is not forfeited

1. Bob creates a withdraw request 100 USDC
2. ST goes up 10%, however the reinvestment fails (idle ST goes up by 10 dollar)
3. Bob receives 110 dollar from the withdrawal because the floor nav at execution is ~100 not 110 because it excluded the idle/ failed reinvested amount (10 dollar), so it calculated yield as: **100-100=0 yield**

Scenario 2: Existing LT premium is incorrectly forfeited

1. Bob creates a withdraw request worth 110 USD, however 10 USD sits in idle, navAtRequest is snapshotted at ~100 not 110 because it excluded the idle premium
2. 10 USD gets succesfully reinvested.
3. Bob receives 100 dollar from the withdrawal because the nav at execution is ~110 because it included the idle/ failed reinvested amount (10 dollar). It calculated yield as: **110-100=10 yield**

Remediation:

Consider adding the value of the idle assets to **navAtRequest** and the nav at execution.

ROYCO5-8 | Address sorting can reverse the E-CLP curve

Fixed

Severity:	Probability:	Impact:
Low	Rare	Medium

Path:

src/factory/templates/liquidity-tranche/BalancerV3_GyroECLP_LT_DeploymentTemplate.sol: _createBalancerV3Pool#L382-L415

Description:

deployMarket calls **_createBalancerV3Pool** to create the ST/quote-asset pool used by the liquidity tranche. The helper sorts the two tokens by address, as required by Balancer, and then creates the pool with the configured E-CLP parameters. This affects every market deployed through this template.

E-CLP parameters are directional: **alpha** and **beta** describe the price of token0 in token1. The current parameters assume that ST is token0 and the quote asset is token1. The pool should either keep that ordering or use a fully transformed parameter set when the tokens are reversed.

However, **_createBalancerV3Pool** sorts the tokens without adjusting either the base or derived E-CLP parameters:

```
(address token0, address token1) = uint160(_seniorTranche) < uint160(_p.quoteAsset)
    ? (_seniorTranche, _p.quoteAsset)
    : (_p.quoteAsset, _seniorTranche);

balancerV3Pool = BALANCER_V3_POOL_FACTORY.create(
    /* ... */
    _p.eclpParams,
    _p.derivedEclpParams,
    /* ... */
);
```

If the ST address sorts above the quote asset, the curve instead prices the quote asset in ST. The effective ST price band is then effectively reversed. The pool would value 1 ST at a value of more than 1 quote asset.

Even after a 1 bp swap fee, an attacker can still sell ST for roughly more quote per unit of NAV and take the difference from LT holders. The reversed curve also changes the stable share rate, leaving much less stable liquidity available for exits.

An attack can happen as follows:

1. A market is deployed where the predicted ST address sorts above the quote asset, so the pool registers **[quote, ST]**.
2. The pool is initialized with quote assets or later receives quote-heavy LT liquidity.
3. An attacker obtains ST through the normal public entry point and sells it into the pool.
4. The reversed curve pays more than ST's NAV even after fees, transferring value from LT holders to the attacker until the price is arbitraged back into line.

```
    function _createBalancerV3Pool(
        GyroECLPPoolParams memory _p,
        address _seniorTranche,
        address _rateProvider,
        address _hook,
        bytes32 _salt
    )
        internal
        returns (address balancerV3Pool)
    {
        // Balancer V3 requires a pool's tokens registered in ascending address order
        (address token0, address token1) = uint160(_seniorTranche) < uint160(_p.quoteAsset) ?
(_seniorTranche, _p.quoteAsset) : (_p.quoteAsset, _seniorTranche);

        BalancerV3TokenConfig[] memory tokens = new BalancerV3TokenConfig[](2);
        tokens[0] = _buildTokenConfig(token0, _seniorTranche, _rateProvider, _p.quoteAssetRateProvider);
        tokens[1] = _buildTokenConfig(token1, _seniorTranche, _rateProvider, _p.quoteAssetRateProvider);

        address authority = ROYCO_FACTORY.ROYCO_AUTHORITY();
        BalancerV3PoolRoleAccounts memory roleAccounts =
            BalancerV3PoolRoleAccounts({ pauseManager: authority, swapFeeManager: authority, poolCreator:
authority });

        balancerV3Pool = BALANCER_V3_POOL_FACTORY.create(
            _p.name,
            _p.symbol,
            tokens,
            _p.eclpParams,
            _p.derivedEclpParams,
            roleAccounts,
            _p.swapFeePercentage,
            _hook,
            _p.enableDonation,
            _p.disableUnbalancedLiquidity,
            _salt
        );
    }
```

Remediation:

Require the predicted ST address to sort below the quote asset and choose a different market ID when it does not. Alternatively, use and validate a complete E-CLP parameter set for the reversed token ordering, including all derived parameters.

ROYCO5-7 | EntryPoint lets request owners bypass blacklist checks

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

src/entrypoint/RoycoDayEntryPoint.sol:_executeDeposit, _cancelDepositRequest, _executeRedemption, _cancelRedemptionRequest

Description:

The entry point handles delayed deposits and redemptions. Users first queue assets or tranche shares, which the entry point holds until the request is executed or cancelled. Execution then calls the tranche on the user's behalf, and cancellation returns the queued funds to a chosen receiver.

Tranche balance updates are meant to reject any locally blacklisted or sanctioned caller, sender, or receiver. For direct operations, this freezes a holder's shares and prevents a depositor from minting shares for a clean address. The same protection should apply throughout the delayed entry-point flow, including when an account becomes blacklisted after submitting a request.

However, EntryPoint custody hides the request owner from the blacklist. On execution, the entry point calls deposit or redeem itself, and redemptions burn shares owned by the entry point:

```
IRoycoVaultTranche(_tranche).deposit(_assets, _receiver);
IRoycoVaultTranche(_tranche).redeem(userSharesRedeemed, _receiver, address(this));
```

The tranche hook therefore checks the EntryPoint instead of the stored _user. A blacklisted account can queue a deposit for a clean receiver, while an account blacklisted after queuing a redemption can still cancel it to a clean address or redeem it for underlying assets. This lets the full queued position escape a freeze that would block the same action if done directly.

One example is:

1. Alice queues her tranche shares through requestRedemption while she is not blacklisted, transferring them to the entry point.
2. Alice is blacklisted during the redemption delay.
3. Alice calls cancelRedemptionRequest(nonce, Bob), where Bob is a clean address.
4. _cancelRedemptionRequest transfers the shares from the entry point to Bob. The blacklist only sees the entry point as caller and sender, so Alice is never checked.

Alternatively, once the delay expires, Alice or a third-party executor can call executeRedemption. The entry point burns its escrowed shares and sends the underlying assets to the request receiver without checking Alice's current blacklist status.

```
    function preTrancheBalanceUpdateHook(
        address _caller,
        address _from,
        address _to,
        uint256 _value
    )
        external
        override(IRoycoDayKernel)
        onlyTranche
        whenNotPaused
    {
        // Get the Royco kernel state
        RoycoDayKernelState storage $ = _getRoycoDayKernelStorage();

        // Batch screen the involved accounts against the market's blacklist
        BlacklistLogic._enforceNotBlacklisted($, _caller, _from, _to);

        // If transferring shares, ensure that the recipient is a whitelisted LP for the tranche
        // The kernel, the protocol fee recipient, and any market-specific tranche share custodian are exempt from this check
        if (
            ENFORCE_TRANCHE_WHITELIST_ON_TRANSFER && _to != address(0) && _to != address(this) && _to != $.protocolFeeRecipient
                && !_isTrancheShareCustodian(_to)
        ) {
            // It is assumed that the sender is already a whitelisted LP
            address authority = authority();
            // Check if the to address can call the deposit function on the tranche
            /// @dev msg.sender is the tranche address
            (bool isWhitelistedTrancheLP,) = IAccessManager(authority).canCall(_to, msg.sender, IRoycoVaultTranche.deposit.selector);
            require(_to != authority && isWhitelistedTrancheLP, ACCOUNT_NOT_WHITELISTED_TRANCHE_LP(_to));
        }

        // Call the market specific pre-balance update hook
        _preTrancheBalanceUpdate(_caller, _from, _to, _value);
    }
```

Remediation:

Check the request owner when accepting, executing, and cancelling each request.

hexens x ROYCO